UNIVERSIDADE FEDERAL DO PIAUÍ
Departamento de Computação

Relatório Técnico

Trabalho de apoio computacional ao projeto de tese

Desenvolvimento de modelos de inteligência artificial para predição de sepse tardia em Unidade de Terapia Intensiva Neonatal

Aluno: Caio Victor Ferreira do Nascimento
Professor: Prof. Dr. Vinícius Ponte Machado

Teresina, 31 de outubro de 2026

1 Objetivo

Este relatório resume o que foi desenvolvido até agora no pipeline de aprendizado de máquina do projeto de tese. Ele lista os requisitos que o projeto define, mostra como cada um foi implementado e indica o que ainda falta. As páginas citadas são as do documento do projeto.

O trabalho concentrou-se na parte do pipeline que **não depende dos dados**: algoritmos, hiperparâmetros, tratamento do desbalanceamento, validação cruzada, avaliação e explicabilidade. O código está no repositório, no ramo `create-pipeline`, e tem 79 testes automáticos, todos aprovados.

2 Requisitos implementados

A Figura 1 mostra o caminho dos dados no pipeline. Cada caixa traz o arquivo responsável, e cada seta, o que esse arquivo grava. A caixa tracejada, dos dados reais, ainda não existe.

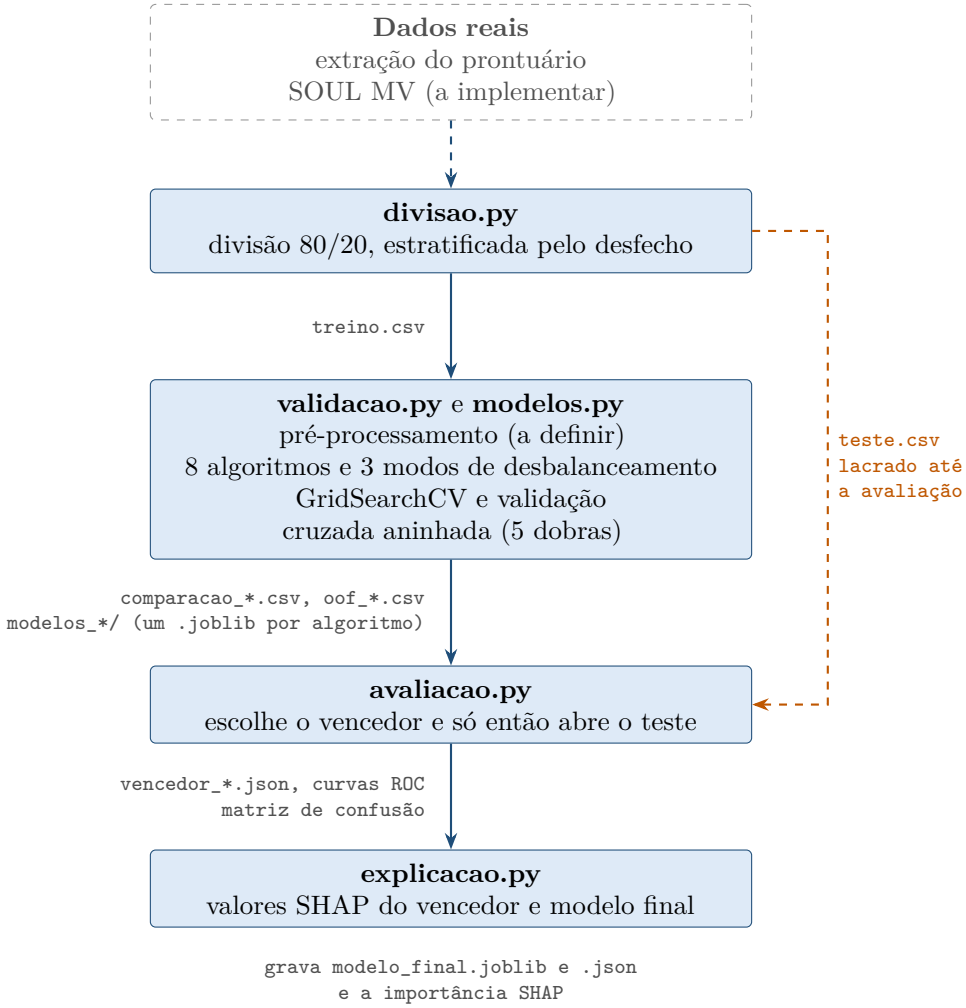


Figura 1: Fluxograma do pipeline, com o arquivo responsável por cada etapa e o que cada um grava.

2.1 Oito algoritmos de classificação

O Quadro 4 (p. 44) lista oito algoritmos: Regressão Logística (RL), Naive Bayes (NB), K-vizinhos mais próximos (KNN), Árvore de Decisão (DT), Random Forest (RF), Gradient Boosting (GB), XGBoost e rede neural MLP. Todos estão no arquivo `modelos.py`, que funciona como um catálogo.

Para cada algoritmo, ele define o estimador, os parâmetros fixos e as grades de hiperparâmetros. Sete vêm da biblioteca scikit-learn, e o XGBoost vem da biblioteca xgboost.

Cada algoritmo é montado num *pipeline* com um espaço reservado para o pré-processamento, sempre antes do modelo. Esse espaço fica vazio até os dados reais chegarem.

2.2 Busca de hiperparâmetros

O projeto prevê a busca com `GridSearchCV` (p. 45). Cada algoritmo tem uma grade enxuta, para os testes, e uma grade completa, prevista para os dados reais, com 59 e 325 combinações no total. Testes automáticos treinam cada algoritmo com a primeira, a do meio e a última combinação de cada grade. Assim, um hiperparâmetro inválido aparece antes de uma busca longa, e não no meio dela.

2.3 Divisão entre treino e teste

O arquivo `divisao.py` lê a tabela, confere a coluna de identificação e o desfecho 0/1 e separa 80% dos pacientes para treino e 20% para teste (p. 45). A divisão é estratificada: as duas partes mantêm a mesma proporção de sepse. O teste fica lacrado até a avaliação final.

2.4 Validação cruzada com cinco dobras

O arquivo `validacao.py` faz a validação cruzada com cinco dobras (p. 45), usando `StratifiedKFold`. A busca de hiperparâmetros escolhe o melhor modelo justamente pelo desempenho nas dobras, então a nota dela é otimista. Por isso, a comparação entre algoritmos usa a validação aninhada da Figura 2: a busca inteira acontece dentro de cada rodada, e o modelo escolhido é medido numa dobra que ele nunca viu.

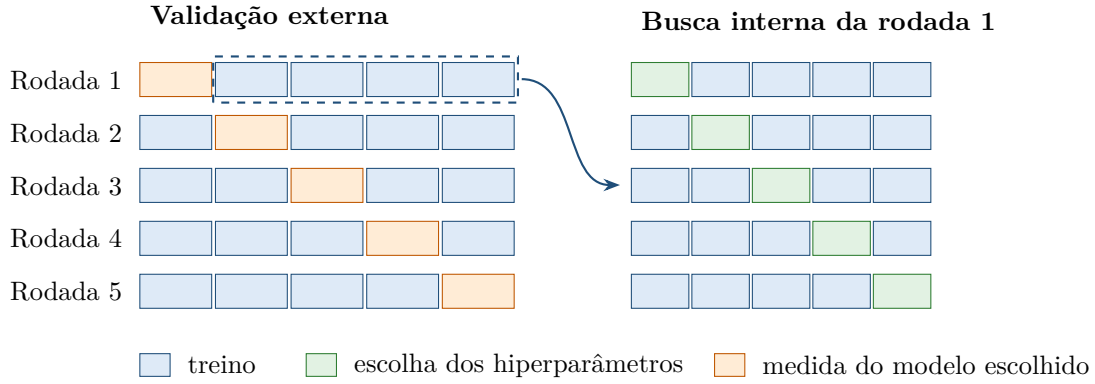


Figura 2: Validação cruzada aninhada. Em cada rodada externa, os quatro quintos de treino são redivididos em cinco dobras para escolher os hiperparâmetros, e o modelo escolhido é medido no quinto restante. O conjunto de teste fica fora de todo o processo.

2.5 Tratamento do desbalanceamento

O projeto prevê pesos de classe e reamostragem sintética (p. 46), porque os casos de sepse são minoria. São três modos, comparados lado a lado, e o Quadro 1 mostra como cada algoritmo recebe o ajuste. O GB e o XGBoost precisaram de uma classe própria. O XGBoost, por exemplo, só aceita um peso fixo, que, calculado com o treino inteiro, incluiria a dobra de validação. A classe própria recalcula o peso a cada treino.

O SMOTE cria casos de sepse por interpolação entre casos vizinhos e, como age nos dados, cobre os oito algoritmos. Ele roda só nas dobras de treino, como mostra a Figura 3. Um teste

Tabela 1: Como cada algoritmo recebe o ajuste em cada modo de desbalanceamento.

Algoritmo	Sem ajuste	Pesos de classe	SMOTE
RL	—	nativo	SMOTE
NB	—	sem suporte	SMOTE
KNN	—	sem suporte	SMOTE
DT	—	nativo	SMOTE
RF	—	nativo	SMOTE
GB	—	classe própria	SMOTE
XGBoost	—	classe própria	SMOTE
MLP	—	sem suporte	SMOTE

automático confirma que, se ele rodasse antes da divisão, o desempenho ficaria artificialmente alto.

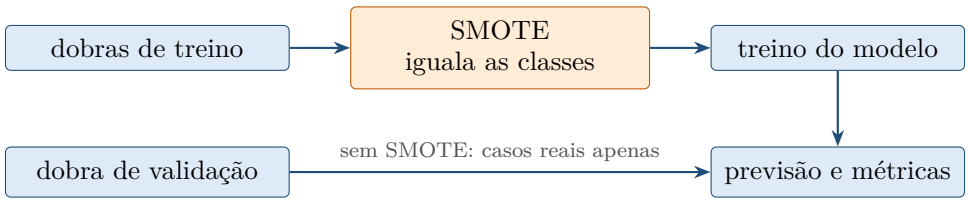


Figura 3: O SMOTE age só no treino de cada dobra. A dobra de validação é avaliada com os casos originais.

2.6 Métricas de desempenho

As métricas do Quadro 5 (p. 45) estão em `validacao.py`: F1-score, acurácia, precisão, sensibilidade e AUROC, além da especificidade, citada no Resumo do projeto. O F1, com limiar 0,5, escolhe o melhor modelo, e a AUROC desempata. Os testes conferem as métricas com contas feitas à mão.

2.7 Comparação em tabela e curvas ROC

O arquivo `avaliacao.py` escolhe o vencedor entre as 24 combinações de algoritmo e modo pela validação aninhada e só depois abre o teste. Em seguida, mede todas as combinações no teste e calcula o intervalo de confiança de 95% do vencedor por *bootstrap*. Gera também a tabela comparativa, as curvas ROC e a matriz de confusão pedidas na p. 46. Ainda não foi executado com dados reais.

2.8 Explicabilidade com SHAP

O arquivo `explicacao.py` calcula os valores SHAP do vencedor (p. 46), isto é, a contribuição de cada variável para a probabilidade prevista de cada paciente. Ele também salva o modelo final com a lista de colunas esperadas e tem a função `prever`, que devolve a probabilidade de sepse de novos pacientes. Ainda não foi executado com dados reais.

3 Por que os dados ainda não foram trabalhados

Os dados reais ainda não existem. A coleta depende da aprovação nos Comitês de Ética e do acesso à API do prontuário SOUL MV e está prevista para 2027 (p. 48). Além disso, as dúvidas da seção 4 mudam quais dados devem ser coletados e como. Por isso, o pré-processamento não foi implementado, e não há dados no repositório.

Os testes automáticos usam uma tabela numérica genérica, criada na memória durante o teste e sem nenhum significado clínico. Ela serve só para verificar que os algoritmos treinam e que nada vaza entre as dobras. Nenhum número dela é resultado do estudo.

4 Dúvidas sobre o projeto

A Figura 4 situa as duas primeiras dúvidas na linha do tempo de uma internação.

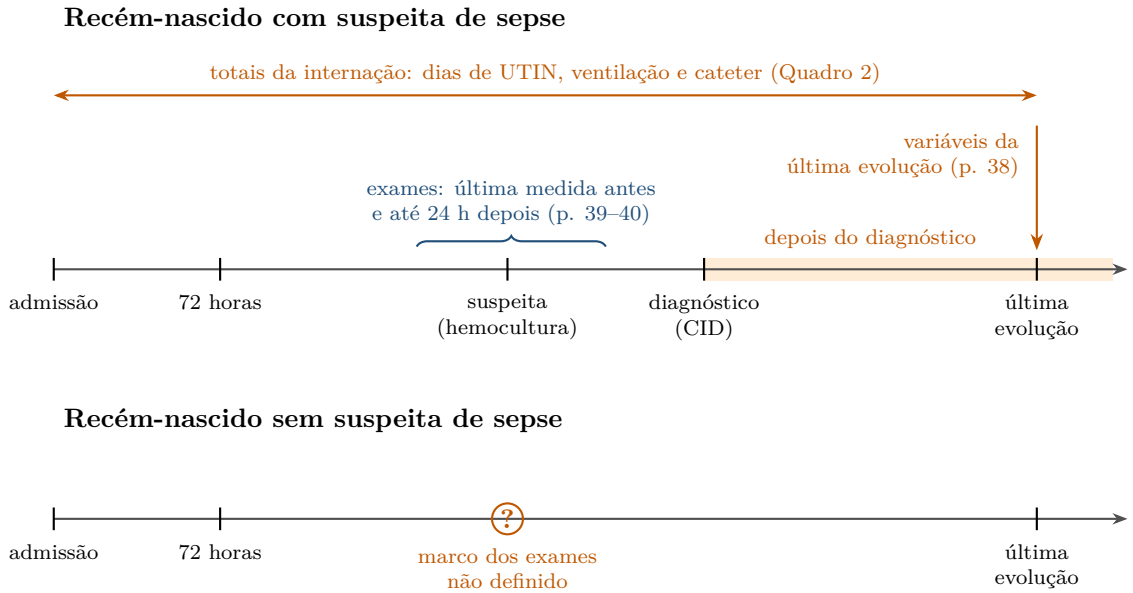


Figura 4: Onde o projeto manda colher as variáveis. A faixa laranja marca o período depois do diagnóstico. Os totais da internação e a última evolução incluem esse período, e, para quem não teve suspeita, não há marco para os exames.

1. **Qual pergunta o modelo responde.** A introdução fala em prever a sepse com antecedência, entre todos os internados (p. 13). Já os exames são colhidos no momento da suspeita (p. 39 e 40), que não existe para quem nunca foi investigado. Se casos e controles tiverem exames de momentos diferentes, o modelo pode aprender a reconhecer quem estava sendo investigado, e não a sepse.
2. **Variáveis que podem ser consequência da sepse.** A última evolução e os totais da internação vêm depois do diagnóstico (p. 38). O resultado da hemocultura e o tempo até o diagnóstico também podem entregar o desfecho ao modelo.
3. **Excluir ou imputar dados ausentes.** A p. 38 manda excluir prontuários incompletos, e a p. 40 manda imputar os valores ausentes.
4. **Escores clínicos.** qSOFA e NEWS2 usam pontos de corte de adultos, que quase todo recém-nascido atinge (p. 33). O nSOFA, feito para neonatos, é citado no texto, mas não está na ficha de coleta.
5. **Critério de sucesso.** O projeto não define o que é “desempenho adequado” (p. 14) nem o limiar de decisão do modelo.

5 O que falta implementar

- Extração dos dados do SOUL MV pela API e cálculo dos escores clínicos (p. 43).
- Pré-processamento, que depende dos dados reais e das dúvidas acima.

- Execução do pipeline completo com os dados reais, incluindo a avaliação final e o SHAP.
- SVM e redes profundas, citados no Quadro 4, com a regra de exclusão abaixo de 70%.
- Análise das variáveis associadas à sepse (objetivo específico 1).

6 Alterações quando os dados chegarem

- Conferir os nomes das colunas de identificação e de desfecho, hoje os do dicionário de variáveis do projeto.
- Implementar o pré-processamento e colocá-lo no espaço reservado do pipeline.
- Excluir ou recortar no tempo as variáveis que dependerem das respostas às dúvidas 1 e 2.
- Rodar a grade completa de hiperparâmetros e revisar seus valores.

Os algoritmos, a validação cruzada, o tratamento do desbalanceamento, a avaliação e o SHAP não precisam mudar.